Machine Learning for Business

BANA 4373 / ECON 4370 · Applied Data Tools

Two-Week Block · Classes 1–4

Dr. Fidel González · Sam Houston State University · Spring 2026

What We Cover This Block

Week 1

Class 1 — The ML Mindset

Prediction vs. inference, ML workflow, train/test split, overfitting, bias-variance

Class 2 — Core Predictive Models

Regularization, decision trees, random forests, gradient boosting, customer churn application

Week 2

Class 3 — Evaluation + Forecasting

Metrics for regression and classification, data leakage, time-series ML, demand forecasting with Prophet

Class 4 — Synthesis

Prediction vs. causation (course arc), explainability, ethics, ML in practice, final project connection

✓ Tip

By the end of this block you will have a **full ML pipeline** from raw features to evaluated predictions — and you will know *when not* to use ML.

Week 1 — Class 1

The ML Mindset

Prediction, Workflow, Overfitting

ML Is Already Running Your Business

Netflix	Amazon	JPMorgan
80%	$35B+	360K hrs
of content watched comes from their recommendation engine. Without ML, most subscribers would churn.	in annual revenue attributed to product recommendation and demand forecasting algorithms.	of legal work automated per year using ML to review loan agreements — in seconds.

The question is not whether your company will use ML. It is whether you will understand it.

The Big Picture: Two Goals, Two Toolkits

Prediction

What will happen?

Will this customer churn?
What price maximizes revenue?
How many units will we sell?

Tools: ML models

This block

vs

Causal Inference

Why does something happen?

Did the ad *cause* more sales?
Did the policy *reduce* crime?
Does training *raise* wages?

Tools: DiD, A/B tests

Last three weeks

△ Watch Out

ML can find that umbrella sales predict rain. It cannot tell you that umbrellas *cause* rain. Causation still requires DiD or experiments.

The ML Vocabulary

Feature (X)	Input variable used to make predictions (age, income, clicks)
Label (y)	What we want to predict (churn? revenue? default?)
Model	The mathematical function $\hat{y} = f(X)$ learned from data
Parameter	Values the model *learns* (OLS coefficients, tree splits)
Hyperparameter	Values *we* set before training (tree depth, learning rate)

Business Translation

You are building a machine that reads customer data (**features**) and outputs a guess (**prediction**) about some business outcome (**label**).

The machine **learns** its internal settings (**parameters**) by studying past examples where you already know the answer.

You control the complexity of the machine via **hyperparameters**.

✓ Tip

Same math, different vocabulary. OLS is ML with one hyperparameter: the functional form is linear.

Supervised vs. Unsupervised Learning

Supervised Learning

You provide labels.

The model learns the mapping $X \to y$ from labeled training examples.

Regression: y is continuous
$\to$ predict revenue, price, demand

Classification: y is a category
$\to$ churn / no churn, fraud / no fraud

Examples: OLS, Lasso, decision trees, random forests, neural networks

Unsupervised Learning

No labels — find structure.

The model discovers patterns without being told what to look for.

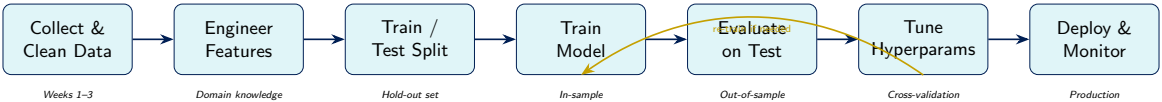
Clustering: group similar observations
$\to$ customer segmentation

Dimensionality reduction: compress features
$\to$ PCA for factor models

Examples: k-means, PCA, DBSCAN

This course focuses on **supervised learning**, *which drives most business ML applications.*

The ML Workflow



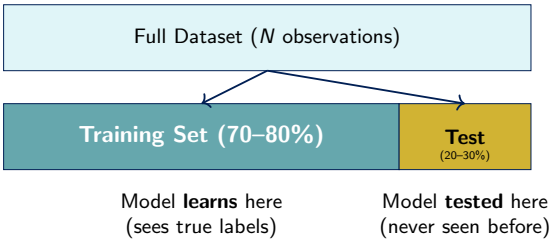
✓ Tip

80% of time in real ML projects is spent on data collection and feature engineering — not on picking the model.

△ Watch Out

Steps are **sequential but iterative**. Poor evaluation forces you back to feature engineering, not just hyperparameter tuning.

Why We Split: The Exam Analogy



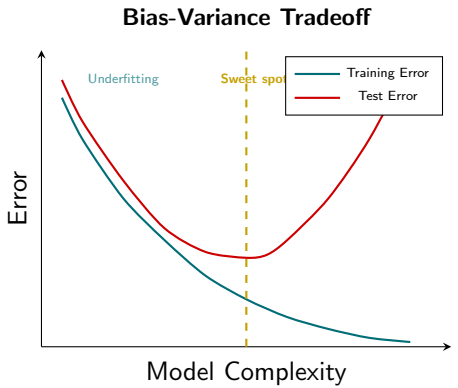
The exam analogy:

- Training set = practice problems
- Test set = the final exam
- A student who memorizes answers to practice problems will fail on new questions

In ML terms:

- A model that memorizes training data will *over-fit* and perform poorly on new customers
- We never let the model see the test set during training
- Test performance = true out-of-sample accuracy

Overfitting: The Central Problem



Underfitting (High Bias):

- Model is too simple
- Misses real patterns
- High error on both train and test
- Example: fitting a line to non-linear data

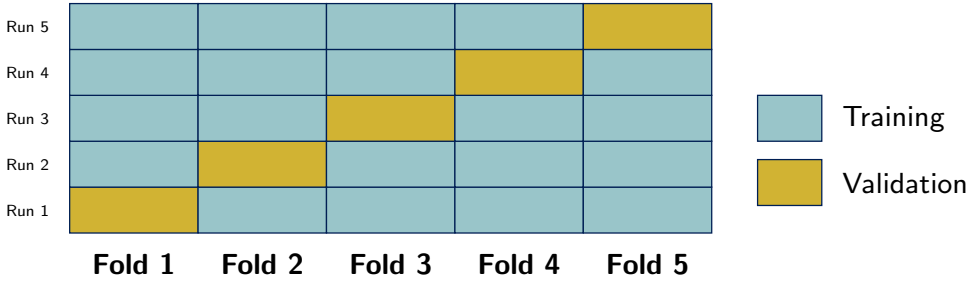
Overfitting (High Variance):

- Model is too complex
- Memorizes noise in training data
- Low train error, high test error
- Example: a tree with depth = 50 on 200 rows

✓ Tip

The **test set gap** is your overfitting detector: train error $\ll$ test error $\Rightarrow$ overfit.

K-Fold Cross-Validation



How it works:

1. Divide data into k equal folds
2. Train on $k-1$ folds, validate on the remaining 1
3. Rotate the validation fold k times
4. Average the k validation scores

Why it matters for business:

- Gives a **stable estimate** of true model performance
- Uses all data for both training and evaluation
- Essential when data is small (common in business settings)
- $k = 5$ or $k = 10$ are standard choices

Write Your Prior — Class 1

⋆ Write Your Prior

Before we start the notebook, answer these on paper:

1. A model achieves **99% accuracy on training data** and **62% accuracy on test data**. What is your diagnosis? What would you do?
2. A bank wants to predict whether a loan will default. List **three features** you think would be useful. List **one feature** that might cause ethical problems.
3. If you use the same data to *train* and *evaluate* a model, will your error estimate be too high, too low, or correct? Explain in one sentence.

Write your answers now — we will revisit them after the notebook.

Class 1 Summary

Key Concepts

- ML solves **prediction** problems; causal inference solves **why** questions
- Supervised learning requires labeled data
- Features (X) $\rightarrow$ model $\rightarrow$ label ($\hat{y}$)
- Always split train and test **before** you touch the data
- Overfitting = memorizing noise; check the train-test gap
- Cross-validation gives a stable error estimate

Coming Up — Class 2

- OLS as our baseline model
- What happens when features explode
- Ridge and Lasso regularization
- Decision trees and random forests
- Applied: customer churn prediction

✓ Tip

Review your **regression notes** from Lecture 9. Class 2 starts from OLS and builds upward.

Week 1 — Class 2

Core Predictive Models

Regularization, Trees, Forests, Boosting

OLS: Your ML Baseline

OLS in ML notation:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}$$

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \boldsymbol{x}_i' \boldsymbol{\beta})^2$$

- **Strengths:** interpretable, fast, works great when $p \ll n$ and relationships are linear
- **Weaknesses:** breaks when features are correlated (multicollinearity), or when p is large relative to n
- **Business context:** always run OLS first. It is your benchmark. Every fancier model must beat it.

When OLS Fails in Practice

Multicollinearity:
You include age, tenure, and experience — all highly correlated. Coefficients become unreliable.

Too many features ($p \approx n$):
Marketing team gives you 200 customer attributes for 300 rows. OLS over-fits perfectly.

Non-linear relationships:
Revenue and advertising spend have diminishing returns — a straight line misses the pattern.

Regularization: Ridge (L2)

Ridge adds a penalty to OLS:

$$\hat{\boldsymbol{\beta}}^{\text{ridge}} = \arg\min_{\boldsymbol{\beta}} \underbrace{\sum_{i=1}^{n}(y_i - \boldsymbol{x}_i'\boldsymbol{\beta})^2}_{\text{fit}} + \underbrace{\lambda \sum_{j=1}^{p} \beta_j^2}_{\text{penalty}}$$

- λ (lambda) controls penalty strength — a **hyperparameter we tune**
- $\lambda = 0 \Rightarrow$ OLS
- $\lambda \to \infty \Rightarrow$ all coefficients $\to 0$
- Coefficients are **shrunk toward zero**, never exactly zero

Business Intuition

Ridge says: "I trust the data, but not too much. If a feature has only weak predictive value, make its coefficient small, not large."

Use Ridge when:

- You have many correlated features (e.g., survey data with 50 Likert items)
- You want to keep all features in the model but need stability

✓ Tip

Always **standardize features** before Ridge. The penalty is scale-dependent.

Regularization: Lasso (L1) — Built-in Feature Selection

Lasso changes the penalty to absolute values:

$$\hat{\boldsymbol{\beta}}^{\text{lasso}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \boldsymbol{x}_i'\boldsymbol{\beta})^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

- The L1 penalty creates **sparse solutions**: many coefficients are driven to **exactly zero**
- This performs **automatic feature selection**
- Very useful when you have hundreds of features and suspect most are irrelevant

	Ridge	Lasso
Penalty	$\sum \beta_j^2$	$\sum \lvert\beta_j\rvert$
Zeros?	Never	Yes
Feature selection	No	Yes

Business Example

A retail chain has **300 product attributes** to predict sales: color, weight, margin, shelf position, region, day-of-week, 250 category dummies...

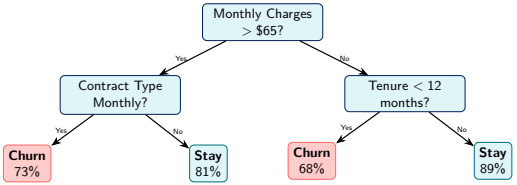
Ridge: uses all 300, shrinks most
Lasso: zeros out 270, keeps 30 key drivers

Lasso gives you a cleaner story for management.

Elastic Net

Combines both: $\lambda_1|\beta| + \lambda_2\beta^2$
Best of Ridge (stability) + Lasso (sparsity).
Common default in practice.

Decision Trees: Splitting the Data



How a tree splits:

1. At each node, find the feature and threshold that best separates the classes (minimizes *impurity*)
2. Common metrics: Gini impurity, entropy
3. Keep splitting until a stopping rule fires (max depth, min samples per leaf)

Pros:

- Highly interpretable
- Handles non-linear patterns
- No feature scaling needed

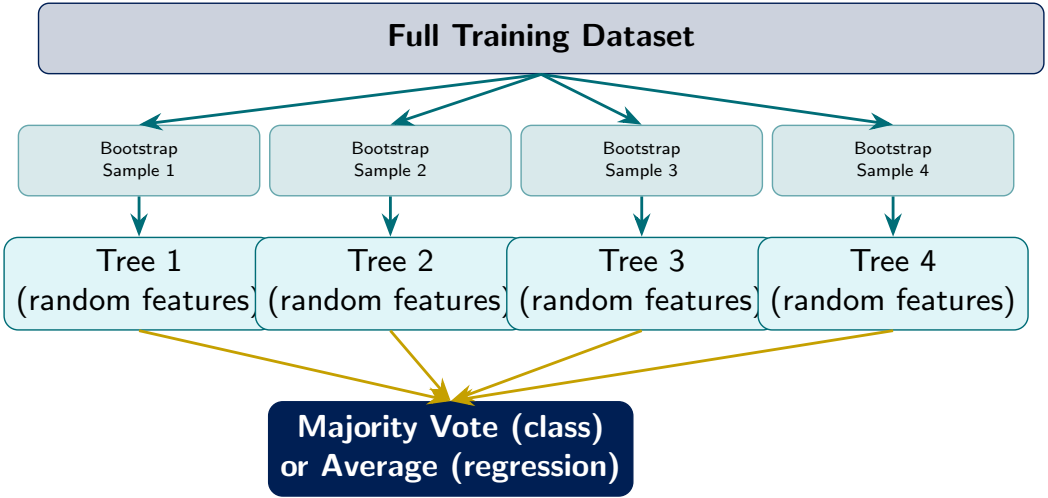
Cons:

- Prone to overfitting
- Unstable (small data changes flip the tree)
- Weak individual predictor

✓ Tip

Trees are rarely used alone. They are the building block of **random forests** and **gradient boosting** — your most

Random Forests: Wisdom of the Crowd



Two sources of randomness:

Gradient Boosting: Learning from Mistakes

Core idea — sequential, not parallel:

1. Fit a shallow tree to the data → Tree 1
2. Compute **residuals** (errors) from Tree 1
3. Fit Tree 2 to the **residuals** (learn what Tree 1 got wrong)
4. Compute residuals from Trees 1+2 combined
5. Fit Tree 3 to those residuals
6. ... repeat for B trees
7. Final prediction = weighted sum of all trees

△ Watch Out

Gradient boosting can overfit if B is too large or trees are too deep. Always tune with cross-validation.

XGBoost, LightGBM, CatBoost

These are optimized implementations of gradient boosting. **XGBoost** wins more Kaggle competitions than any other algorithm and is standard in industry.

Key hyperparameters to tune:

- `n_estimators` (number of trees)
- `max_depth` (tree depth)
- `learning_rate` (shrinkage)
- `subsample` (row sampling)

Business Application: Customer Churn Prediction

The Business Problem

A telecom company loses **$300/customer** when a subscriber cancels. They have 10,000 customers and **15% monthly churn rate**. That is $450,000/month in lost revenue.

Goal: Identify at-risk customers *before* they leave so retention teams can intervene with targeted offers.

Label: Churned (1) or Stayed (0) in the next 30 days

Features: Monthly charges, contract type, tenure, service calls, payment method, data usage, ...

Model comparison on held-out test set:

Model	Accuracy	AUC
Logistic Regression	78%	0.81
Ridge / Lasso	80%	0.83
Decision Tree	77%	0.79
Random Forest	**85%**	**0.89**
Gradient Boosting	**87%**	**0.91**

✓ Tip

Business impact: catching 87% of churners and offering a $30 discount means saving ≈ $380 net per retained customer. Churn models pay for themselves.

Choosing the Right Model: A Decision Guide

Model	Interpretable?	Handles Non-linearity?	Scales to Many Features?	Typical Performance	Start Here When...
OLS / Logistic	High	No	Moderate	Baseline	You need explainability
Ridge / Lasso	High	No	Yes	Good	Many correlated features
Decision Tree	High	Yes	Moderate	Weak alone	Stakeholders need rules
Random Forest	Moderate	Yes	Yes	Very Good	General purpose, robust
Gradient Boost	Low	Yes	Yes	Best	Max predictive accuracy

✓ Tip

Rule of thumb: Always start with OLS/Logistic as your baseline. Upgrade only when performance gains justify the loss of interpretability.

△ Watch Out

No Free Lunch Theorem: no single model outperforms all others on all problems. Choose based on your data, interpretability needs, and business constraints.

Write Your Prior — Class 2

⋆ Write Your Prior

Before opening the churn notebook, write your answers:

1. You are building a churn model. You have these features: customer name, account number, monthly bill, contract type, number of support calls, and zip code. **Which features would you include?** Justify each.
2. Without running any code: do you expect Random Forest or Logistic Regression to perform better on churn prediction? Write a one-sentence reason.
3. A model correctly classifies 90% of customers. The churn rate is 5%. Is 90% accuracy impressive? (Hint: what happens if the model predicts "stay" for everyone?)

△ Watch Out

Question 3 is a trap. You will revisit it in Class 3 when we cover classification metrics. If accuracy alone were enough, we would not need AUC, precision, or recall.

Class 2 Summary

Key Concepts

- OLS is always your **baseline**; everything must beat it
- Ridge shrinks coefficients; Lasso drives some to zero (feature selection)
- Decision trees are interpretable but overfit easily
- Random Forests reduce variance through **bagging** + feature randomness
- Gradient Boosting learns from residuals sequentially
- Choose models based on interpretability vs. performance tradeoff

Coming Up — Week 2, Class 3

- How do we measure model quality fairly?
- Regression vs. classification metrics
- Confusion matrix, ROC, AUC
- **Break the Estimator:** data leakage
- Time-series as a prediction problem
- Demand forecasting with Facebook Prophet

✓ Tip

Homework 6 is now live. Start with the train/test split cell before touching any model.

Week 2 — Class 3

Evaluation + Forecasting

Metrics, Data Leakage, Time-Series ML

Evaluating Regression Models

Three standard metrics:

Mean Absolute Error (MAE):

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Interpretable in original units. Robust to outliers.

Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Penalizes large errors more. Also in original units.

R^2 **(Coefficient of Determination):**

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Fraction of variance explained. Between $-\infty$ and 1.

Business Translation

MAE = $1,200 means your demand forecast is off by $1,200 on average.

RMSE = $1,800 signals there are occasional large misses (the squaring punishes them).

$R^2 = 0.82$ means your features explain 82% of variation in sales.

△ Watch Out

Never use R^2 alone on a test set. Adding random noise can increase in-sample R^2. Use RMSE on the held-out test set.

✓ Tip

Use **MAPE** (Mean Absolute Percentage Error) when you need scale-independent comparisons across products with very different price ranges.

Evaluating Classification Models

The Confusion Matrix:

	Predicted: 1	**Predicted: 0**
Actual: 1	TP (True Pos.)	FN (Miss)
Actual: 0	FP (False Alarm)	TN (True Neg.)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why Accuracy is Not Enough

Churn rate = 5%.
Model that predicts "Stay" for everyone:
Accuracy = 95% !!
Recall = 0% (catches zero churners)

This model is **useless** to a retention team.

Precision vs. Recall Tradeoff

High Precision → few false alarms. Offer discounts only to true churners.

High Recall → catch more churners, but call some loyal customers unnecessarily.

Your business context determines the right balance.

Why We Need Something Beyond Accuracy

Suppose We Predict Customer Churn

Your model gives each customer a **score**:

- Customer A: 0.92
- Customer B: 0.74
- Customer C: 0.41
- Customer D: 0.08

These are **predicted probabilities** of churning.

The Problem

To turn scores into a yes/no prediction, you need a **threshold**:

$$\text{Predict churn if } \hat{p} > c$$

If $c = 0.50$, maybe A and B are labeled "churn."
If $c = 0.80$, only A is labeled "churn."

△ Watch Out

Accuracy depends on the cutoff.

A model can look good or bad depending on where you draw the line.

✓ Tip

AUC solves this problem.

Instead of evaluating the model at *one* threshold, AUC summarizes performance **across all possible thresholds**.

Q&A

Question: Why not just use 0.50?

Because the best cutoff depends on business costs. Missing a true churner may be expensive. Contacting a loyal customer may be annoying but cheap.

AUC Intuition: How Well Does the Model Rank Cases?

Core Idea

AUC asks a ranking question:

If I randomly pick one true positive case and one true negative case,
how often does the model give the positive case a higher score?

Example

Suppose:

- True churner gets score 0.81
- True non-churner gets score 0.33

Good — the model ranked them correctly.

If this happens most of the time, AUC is high.

$$\text{AUC} = P(\text{score of positive} > \text{score of negative})$$

AUC	Meaning
0.50	Random ranking
0.70	Usually right
0.80	Strong ranking
0.90	Excellent ranking
1.00	Perfect ranking

✓ Tip

Important: AUC is not mainly about whether predicted probabilities are perfectly calibrated.
It is about whether the model puts higher-risk observations above lower-risk observations.

△ Watch Out

A high AUC does **not** automatically mean the model is useful. You still need to choose a threshold that fits the business problem.

From Thresholds to the ROC Curve

What Happens When We Move the Threshold?

As we lower the threshold:

- We catch more true positives
- But we also create more false positives

So every threshold creates a different tradeoff.

ROC Curve

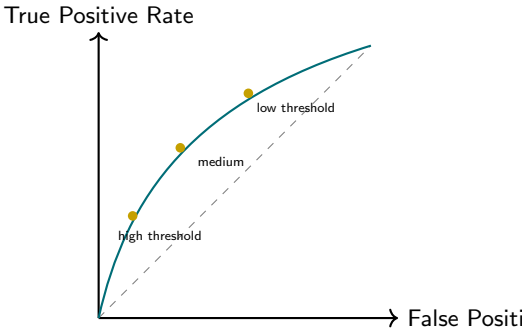
The ROC curve plots:

True Positive Rate vs. False Positive Rate

for **every possible threshold**.

✓ Tip

The ROC curve shows the full menu of tradeoffs.
The AUC summarizes that curve in one number.

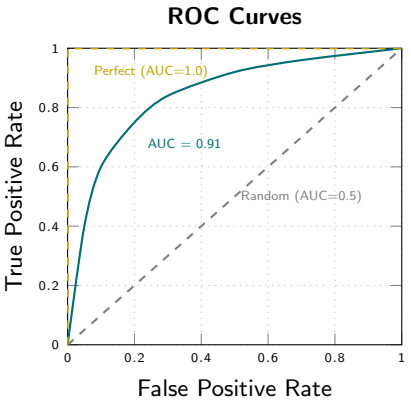


Q&A

Question: Is higher always better?

For AUC, yes. A curve closer to the top-left corner is better because it gets more true positives with fewer false positives.

ROC Curve and AUC



How to read the ROC curve:

- X-axis: False Positive Rate (false alarms)
- Y-axis: True Positive Rate (recall)
- Each point = a different classification threshold
- **AUC = Area Under the Curve**

AUC	Interpretation
0.50	No better than random
0.70	Acceptable
0.80	Good
0.90	Excellent
1.00	Perfect (likely overfit)

✓ Tip

AUC lets you compare models **across all thresholds**. This is especially useful when the best cutoff depends on business costs (calling customers is cheap; giving discounts is not).

Break the Estimator: Data Leakage

Break the Estimator

Scenario: You are building a model to predict which hospital patients will be readmitted within 30 days. Your team achieves $R^2 = 0.97$ on the test set. Everyone is excited. You deploy the model. It completely fails in production. **What happened?**

Types of data leakage:

1. **Target leakage:** A feature contains information that is only available *after* the outcome is realized.
   Example: including "number of follow-up calls" as a feature — only logged after readmission.
2. **Train/test contamination:** Scaling, imputing, or encoding using the full dataset (including test rows) before splitting.
3. **Time leakage:** Using future data to predict the past in time-series contexts.

△ Watch Out

The fix is always the same:

Split first. Preprocess second.

Fit your scaler, imputer, and encoder on **training data only**. Transform test data using training statistics.

Use `sklearn.Pipeline` to enforce this automatically.

Q&A

Q: Is a $R^2 = 0.97$ possible without leakage?
A: Very rarely. Almost always a leakage signal. Investigate aggressively before celebrating.

Time Series as a Prediction Problem

The decomposition framework:

$$y_t = T_t + S_t + C_t + \varepsilon_t$$

- T_t = **Trend:** long-run direction (sales growing over years)
- S_t = **Seasonality:** regular periodic patterns (holiday spikes, weekly cycles)
- C_t = **Cycle:** irregular business-cycle fluctuations
- ε_t = **Noise:** unexplained random variation

ML approach to time series:

- Create **lag features**: $y_{t-1}, y_{t-2}, \ldots$
- Add **calendar features**: month, day of week, is_holiday
- Add **rolling statistics**: 7-day moving average, 30-day std
- Then apply any ML model to the feature matrix

△ Watch Out

textbfNever shuffle time series data!

Standard train/test split randomly shuffles rows — this would let the model "see the future" during training.

Always split **chronologically**: train on earlier periods, test on later periods. In scikit-learn use `TimeSeriesSplit`.

Business Forecasting Questions

- How many units will we sell next month?
- What will our website traffic look like next week?
- When will demand peak during the holiday season?

Facebook Prophet: Business-Ready Forecasting

What Prophet does:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t$$

- $g(t)$ = **Trend:** linear or logistic growth, with automatic changepoint detection
- $s(t)$ = **Seasonality:** weekly, monthly, yearly — Fourier series
- $h(t)$ = **Holidays:** built-in US holiday effects; add custom ones (Black Friday, back-to-school)

Why business teams love it:

- Handles missing data and outliers gracefully
- Produces **uncertainty intervals** automatically
- No time-series expertise required
- 5 lines of Python; readable output for management

Prophet in 5 Lines

```
from prophet import Prophet
m = Prophet(
  yearly_seasonality=True,
  weekly_seasonality=True)
m.fit(df) # df has ds, y cols
future = m.make_future_dataframe(
  periods=90)
forecast = m.predict(future)
m.plot(forecast)
```

✓ Tip

Prophet is not a black box — its components are fully **decomposable and plotable**, making it easy to explain seasonal patterns and trend changes to business stakeholders.

Business Application: Retail Demand Forecasting

The Business Problem

A national retailer stocks 50,000 SKUs across 200 stores. Overstocking costs \$0.30/unit/week in storage. Understocking costs \$2.50/unit in lost margin.

Goal: Forecast weekly unit sales 4 weeks out for each store-SKU combination to optimize inventory ordering.

Traditional approach: use last year's same week (naive seasonal benchmark)

ML approach: lag features + calendar effects + price + promotions + weather → XGBoost or Prophet

Model results (4-week horizon, held-out test):

Model	MAPE	Improvement
Naive (last year)	28%	—
ARIMA	21%	−7 pp
Random Forest	15%	−13 pp
XGBoost	**11%**	−17 **pp**
Prophet + Regressors	13%	−15 pp

✓ Tip

A 17 pp improvement in MAPE on a \$50M inventory budget translates to millions in freed working capital. This is how ML creates measurable business value.

Write Your Prior — Class 3

⋆ Write Your Prior

Before running the forecasting notebook:

1. A retailer sees weekly sales data for 3 years. You want to train a model to forecast the next 4 weeks. **Describe** how you would split the data. Would you split randomly? Why or why not?
2. You train a Prophet model on 3 years of data. It achieves MAPE = 4% on training data and MAPE = 22% on the test period. What does this tell you? What would you check first?
3. Name **two external features** beyond historical sales that might improve a retail demand forecast. Why would each help?

Class 3 Summary

Key Concepts

- Regression: use **RMSE and MAE** (in units), not R^2 alone
- Classification: **AUC** $>$ Accuracy when classes are imbalanced
- **Data leakage** produces unrealistically high scores; always split before preprocessing
- Time series requires **chronological** splits, lag features, and calendar effects
- **Prophet** decomposes trend, seasonality, and holiday effects — interpretable and deployable

Coming Up — Class 4

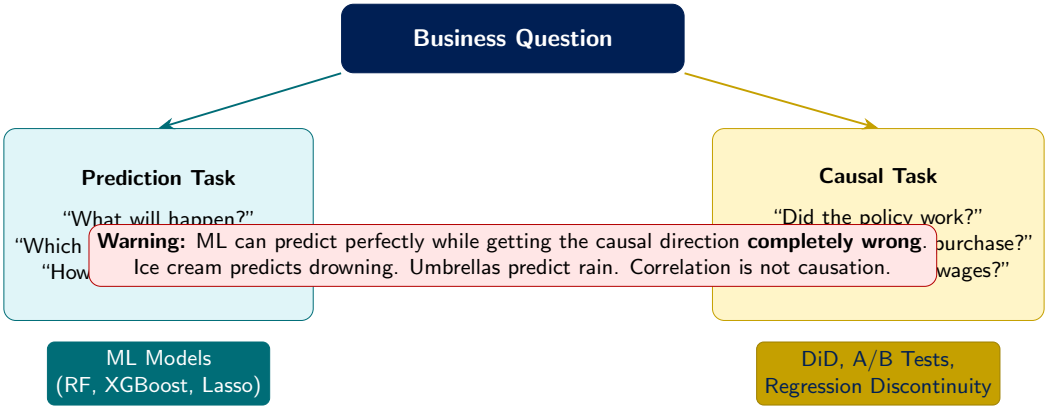
- Prediction vs. causation: the final answer
- When to use ML, when to use DiD/A-B tests
- Model explainability (SHAP values)
- Fairness and ethical considerations
- The full course toolkit, end to end
- Final project alignment

Week 2 — Class 4

Prediction vs. Causation

Synthesis, Explainability, and the Course Arc

The Central Question: Predict or Explain?



Which Tool Do You Need? A Decision Matrix

Business Question	Right Tool	Why
Will this customer default on their loan?	ML (Gradient Boosting)	Pure prediction; no policy action needed
Did our price change *cause* fewer sales?	DiD / Regression Discontinuity	Need causal effect, not just correlation
Does the new checkout UX *increase* conversions?	A/B Test	Randomization eliminates confounding
How many units will we sell next quarter?	ML Forecasting (Prophet/XGBoost)	Prediction problem with time structure
Which features explain customer satisfaction?	OLS / Lasso	Interpretability + inference on coefficients
Did the training program *raise* employee productivity?	DiD (pre/post, treatment/control)	Observational causal inference

✓ Tip

The skill is not knowing one tool — it is knowing **which tool to reach for** given the question. This is what distinguishes a data scientist from a model-runner.

Opening the Black Box: Feature Importance and SHAP

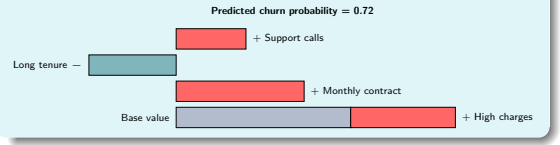
Why explainability matters:

- Regulators require explanations for credit decisions (Fair Credit Reporting Act, EU AI Act)
- Management needs to trust and understand the model
- Debugging: high-importance features that surprise you often signal data leakage or a flawed setup
- Customer-facing decisions require audit trails

SHAP (SHapley Additive exPlanations):

- From game theory: how much does each feature **contribute** to a prediction *for a specific observation*?
- Works for any model (trees, neural nets, Lasso)
- Produces both global (overall) and local (per-customer) explanations
- Standard in industry; `pip install shap`

SHAP Waterfall — One Customer



✓ Tip

SHAP lets you tell a customer: "Your high monthly bill and month-to-month contract are the main risk factors."

Fairness and Ethics in Business ML

Three failure modes to know:

1. **Proxy discrimination:**
   Zip code predicts creditworthiness. Zip code is correlated with race. The model is discriminatory even without using race as a feature.

2. **Feedback loops:**
   A model trained on historical hiring decisions learns to penalize women if they were historically underrepresented in promoted roles.

3. **Measurement bias:**
   Using arrests to predict recidivism conflates actual crime with policing intensity in certain neighborhoods.

△ Watch Out

Real cases:
Amazon scrapped a hiring ML model (2018) that downgraded resumes mentioning "women's."

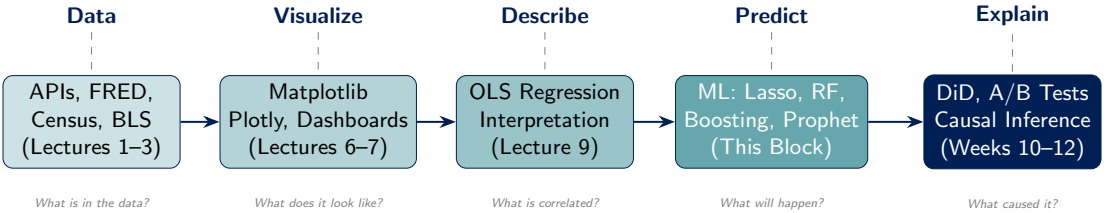
COMPAS recidivism tool was found to produce racially disparate false positive rates.

Apple Card (2019) received regulatory scrutiny for offering lower credit limits to women than men with similar profiles.

✓ Tip

Practical check: after building any model, slice your error metrics by demographic group. If AUC differs significantly across groups, investigate before deploying.

The Full Toolkit: Your Course Arc



You now have the complete applied data science toolkit for business.

What You Can Now Do

You can now collect, clean, visualize, and model data.

You can predict outcomes with ML.
You can estimate causal effects with DiD and experiments.
You can communicate results to a business audience.

That is a complete, job-ready applied data science skillset.

Predict	Explain	Decide
Will a customer churn?	What drives the outcome?	Did the intervention work?
How much will we sell?	Which features matter?	What caused the change?
Is this transaction fraud?	Is the model fair?	What should we do next?

Class 4 and Block Summary

Four-Class Block Summary

- **Class 1:** ML mindset, prediction vs. inference, train/test split, overfitting
- **Class 2:** OLS baseline, regularization, decision trees, random forests, gradient boosting
- **Class 3:** Evaluation metrics (RMSE, AUC), data leakage, time-series ML, Prophet forecasting
- **Class 4:** Prediction vs. causation synthesis, SHAP explainability, fairness, full course arc

Remaining Course Milestones

- **HW 6** due April 27 — ML pipeline notebook
- **Final Project Proposal** due April 20
- **Final Presentations** May 6, 10:15 am – 12:15 pm, SHB 202
- **Final Repo** due May 6 at midnight

✓ Tip

Office hours: Mon & Wed 11 am – 12 pm, SHB 241-A. GroupMe is the fastest way to get help on HW 6.

Questions? See you for final project presentations!